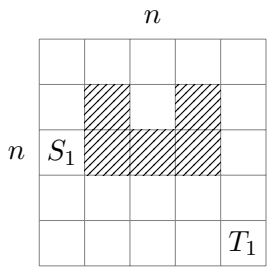
# Network Flow, Ford-Fulkerson Algorithm & Max-Flow Min-Cut Theorem

Instructor: Sourav Chakraborty. Scribed by: Arkajit Ganguly (CrS2504) , Abhijit Santra (CrS2502)

13.04.26

## 1 Shortest Path on a Grid

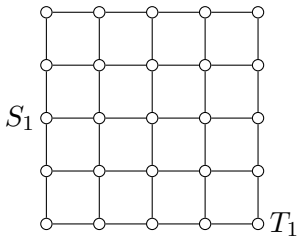
Consider an $n \times n$ grid where some cells are blackened (impassable). Given a source $S_1$ and target $T_1$, find the shortest path from $S_1 \to T_1$.




**Observation:** With a greedy approach, you might get stuck in a local minimum.

### Approach: Encode as a Graph

Encode the $n \times n$ grid as a graph (cells as nodes; blackened cells aren't there).




Now you can use any shortest path algorithm. (Everything is **NOT** a heuristic.)

## 2 Multiple Sources and Targets

### Problem 1: Pairwise Sources and Targets

Suppose we have multiple starting points and we want $S_i \to T_i$ (let's say $K$ many).

We'll run a shortest path algorithm $K$ times:

$$\text{Complexity: } K(|V| + |E|)$$

For large $K$:

$$|V|(|V| + |E|)$$

**Another approach:** Use *all-pairs shortest path.*

### Problem 2: Any Source to Any Target

Again, changing the problem: we have a set of sources ($S$) and a set of terminals ($T$) (like fire brigades and burning places).

Here, I don't want $S_i \to T_i$. I'm happy with $S_i \to T_k$ for any $i$ and any $k$.

**One approach:** Look at all possible permutations.

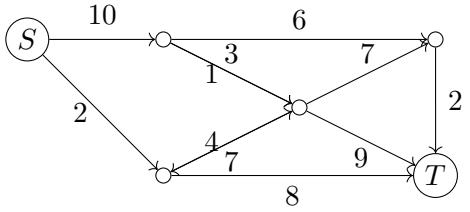
Complexity becomes $K!$ (problem!)

| All perms | |
|---|---|
| $S_1$ | $t_1$ |
| $\vdots$ | $\vdots$ |
| $S_K$ | $t_K$ |

Let's talk about a different problem which offers a much more powerful modelling (seemingly unrelated).

## 3 Network Flow

**NETWORK FLOW:** just a directed graph where weights are *capacities* of the edges.




$$G = (V, E, c : E \to \mathbb{R}^+)$$

The source has **UNLIMITED SUPPLY**; the target has $\infty$ **absorbing provision**.

### Definition: Flow

Find a flow $F$ where $F : E \to \mathbb{R}^+$ such that:

(I) $F(e) \leq c(e) \quad \forall e \in E$ (# shouldn't exceed the capacity)

(II) **[Kirchhoff's Law]** For every $v \in V \setminus \{S, T\}$:

$$\underbrace{\sum_u F(u, v)}_{\text{flow coming in } v} = \underbrace{\sum_w F(v, w)}_{\text{flow going out from } v}$$

### Goal: Maximize the Flow

$$\max \sum_u F(S,u) = \sum_u F(u,T)$$

*[This equality will hold due to Kirchhoff's Law!]*

### Linear Programming Formulation

Given a graph with capacities, can you devise an algorithm that maximizes the FLOW?

Treat the unknown flow values as variables:

$$\begin{aligned} \max \quad & \sum_u F(S,u) \\ \text{s.t.} \quad & \forall (u,v) \;\; F(u,v) \leq C(u,v) \\ & \forall v \;\; \sum_u F(u,v) = \sum_u F(v,u) \\ & F(u,v) \geq 0 \in \mathbb{R} \end{aligned}$$

This is a **linear programming problem (LPP)** – can be done in polynomial time.

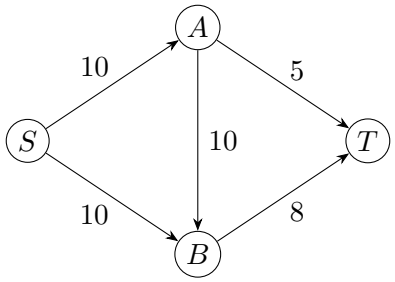
## 4 A Greedy Approach

The problem is more interesting than the LPP solution itself. When we solve the problem on some graph, the flow may not be in integers.

**NETWORK FLOW** can be solved through LP (algorithm not given). We'll learn an algorithm from scratch to do this problem.

### Example: First, be Greedy

Total capacity $\leftarrow 30$.




**First path:** $S \to A \to B \to T$, send 5 units.

$$S \xrightarrow{5} A \xrightarrow{5} B \xrightarrow{5} T$$

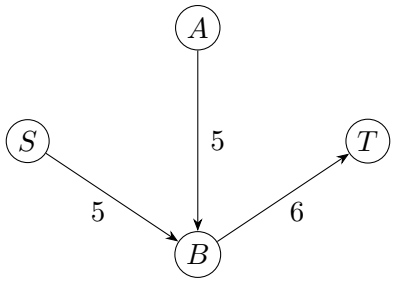
Remaining capacity: $5, 5, 6$ on the respective edges (with new capacities $\boxed{5}, \boxed{5}, \boxed{6}$).

**Second path:** $S \to A \to T$, send 5 units.

$$S \xrightarrow{5} A \xrightarrow{5} T$$

Remaining capacity: $\boxed{0}, \boxed{5}, \boxed{6}$.

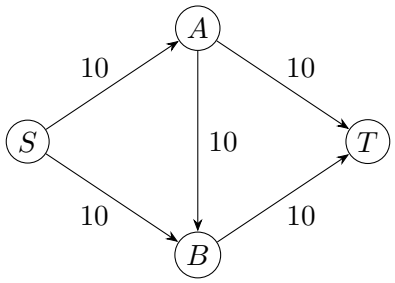
**I'm left with the RESIDUAL GRAPH.**



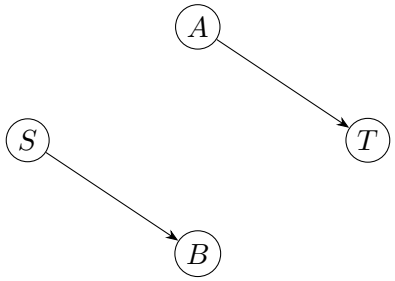

**NO PATH left** – this is my flow.

## 5 Greedy is NOT Optimal

We shall show the last approach is **NOT optimal**.



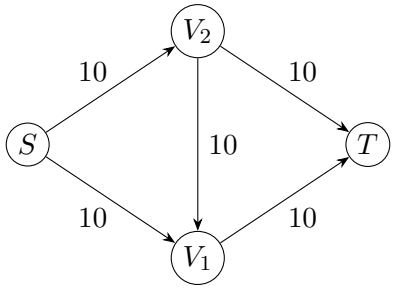

If we do $\{S \xrightarrow{10} A \xrightarrow{10} B \xrightarrow{10} T\}$ – only passes **10 units** (suboptimal).
In the residual graph, clearly we could have pushed **20 units**!




### Insight

The future choices are "bounded by" the quality of the initial choice.

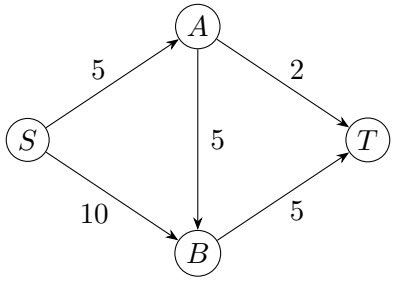
### Analogy: Backward Channel



$S$ has to pay: $V_1 \to 10K$ INR, $V_2 \to 10K$ INR.

**Key Idea:** *Once a flow exists, there is now a* ***backward channel*** *created.*

# 6 Backward Channels and Residual Graph




Consider $S \to A \to B \to T$ with capacities $5, 5, 5$.

After sending flow along this path, capacities become: $\boxed{0}, \boxed{1}, \boxed{0}$ (in the residual sense; the difference reflects the bottleneck).

### Algorithm Idea

**Try to find a path. Try to send as much as you can.**

But in the leftover (residual) graph:

- The capacity goes **down** ($\downarrow$).
- But the **backward channel** capacity goes **up** ($\uparrow$).

**Claim:** Whatever flow $F$ I create at any point in time by the above algorithm will be a valid one (satisfying Kirchhoff's Law).
*(Prove!)*

# 7 Ford-Fulkerson Algorithm

## Pseudocode

**Algorithm 1** Ford-Fulkerson Algorithm

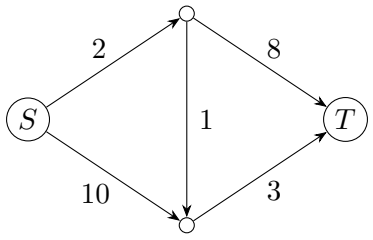
```
1:  while ∃ a path P : S → T in G do
2:      f ← min_{e∈P} c(e)
3:      for each e ∉ P do
4:          F(e) = F(e)                                   ▷ unchanged
5:      end for
6:      for each e ∈ P do
7:          F(e) = F(e) + f
8:          F(u, v) = −F(v, u)
9:      end for
10:     for each e ∉ P do
11:         c(e) = c(e)                                   ▷ unchanged
12:     end for
13:     for each (u, v) ∈ P do
14:         c(u, v) = c(u, v) − f
15:         c(v, u) = c(v, u) + f                         ▷ backward channel
16:     end for
17: end while
```

**Prove:** This creates a valid flow (Kirchhoff's Law is satisfied).
This is known as the **FORD-FULKERSON ALGORITHM!**

## Two Claims

- **Claim 1:** $F$ is a valid FLOW.
- **Claim 2:** Every iteration, the FLOW is increasing by at least 1.

## Termination Argument




The FLOW is upper-bounded by $2 + 10 = 12$.

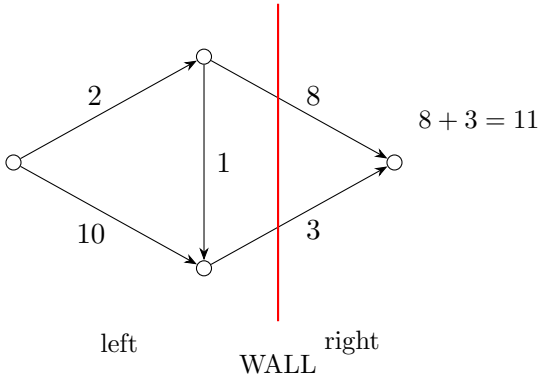
Since $\sum_u F(S, u)$ increases by 1 every round, by the **Infinite Descent Principle of Fermat**, the algorithm STOPS.

# 8 Integral Flows and Cuts

### Claim (Integrality)

If $c : E \to \mathbb{N}$ (integer capacities), then the flow $F$ is also an integer.

### Better Upper Bound: The Cut




**Question:** How much of produced water goes from *left* to *right*?

$$\text{Sum of capacities (left, right)} = 2 + 3 = \boxed{5}$$

### What's the wall here? It's a CUT.

An $ST$-**cut** partitions the vertices: $V = U_S \uplus U_T$, with $S \in U_S$, $T \in U_T$.

$$\boxed{\sum_{\substack{u \in U_S \\ v \in U_T}} c(u, v)} \quad \longleftarrow \text{ is an UPPER BOUND over the flow!}$$

The minimum value:

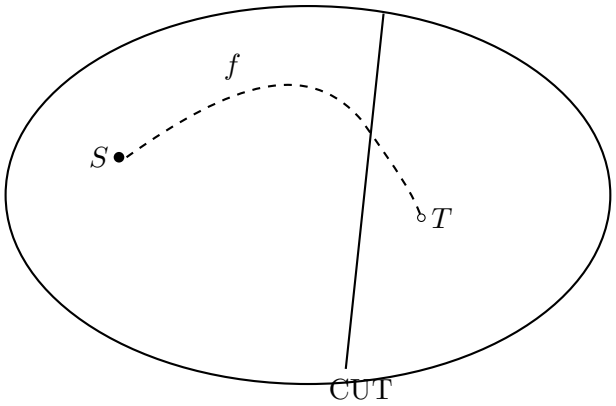
$$\min \sum_{\substack{u \in U_S \\ v \in U_T}} c(u, v) = \text{min-CUT}$$

is an **UPPER-BOUND** on the MAX FLOW.

$$\implies \text{maxflow} \leq \min_{ST\text{-cut}} \sum_{\substack{u \in U_S \\ v \in U_T}} c(u, v)$$

# 9 Max-Flow Min-Cut Theorem

**Prove that:** The min-cut **decreases** by $f$ (the flow pushed along the augmenting path).




at every cross:

$\rightarrow$ K times forward

you'll need to do

$\rightarrow (K-1)$ times backwards

The algorithm halts when the cut $= 0$, and max-flow hits its maxima:

$$\boxed{\text{maxflow} = \min_{ST\text{-cut}} \sum_{\substack{u \in U_S \\ v \in U_T}} c(u,v)}$$

**MAX-FLOW MIN-CUT THEOREM** $\Uparrow\Uparrow$

## 10 Summary

We looked at:

(I) **Network Flow**.

(II) **Algorithm Prototype** (vaguely choosing a path)

- Stops in polynomial time.
- FORD-FULKERSON Algo.

(III) **Capacities are integers** $\Longrightarrow$

- INTEGRAL Solution.
- Max-flow = Min-cut.

(IV) **Exercise:** Prove Hall's Marriage Theorem using max-flow min-cut.